

Simulateur de trajets taxis

Owner	 louis reynouard
Tags	

Présentation

Le simulateur est un script Python autonome qui génère des fichiers Parquet dans votre Volume Unity Catalog, imitant l'arrivée de nouveaux trajets taxis en temps réel. Il est la colonne vertébrale de tous les TP de ce cours.

Il ne nécessite pas de Kafka, de broker de messages, ni de service externe. Il tourne dans un thread Databricks, à côté de vos notebooks.

Architecture du simulateur

```
taxi_simulator.py
├── TaxiStreamSimulator (classe principale)
├── TripGenerator       (génère des trajets selon le mode)
└── ChaosInjector       (injecte des données corrompues en mode chaos)
```

Le simulateur génère des trajets avec des distributions calibrées sur le dataset NYC Taxi 2022-2024 : zones pondérées par fréquence réelle, montants en distribution normale, distribution horaire avec pointes 7-10h et 17-20h.

Installation

Déposez `taxi_simulator.py` dans votre Volume :

```
# Dans un notebook, cellule 1
import subprocess
subprocess.run([
    "cp",
```

```
    "/Volumes/workspace/tp_spark_XYZ/scripts/taxi_simulator.py",  
    "/tmp/taxi_simulator.py"  
])
```

Ou uploadez-le directement via l'interface Databricks : Catalog → Volume → Upload to this volume.

Paramètres

Paramètres de base

Paramètre	Type	Défaut	Description
<code>output_path</code>	str	obligatoire	Chemin du Volume où écrire les fichiers Parquet
<code>rate</code>	int	20	Nombre de trajets générés par batch
<code>interval_seconds</code>	int	30	Durée entre deux batches (en secondes)
<code>mode</code>	str	"normal"	Mode de génération (voir ci-dessous)
<code>duration_batches</code>	int	20	Nombre de batches avant arrêt automatique

Paramètres avancés

Paramètre	Type	Défaut	Description
<code>late_data_ratio</code>	float	0.0	Fraction de trajets avec Event Time en retard (0.0 à 0.5)
<code>late_data_max_minutes</code>	int	10	Retard maximum en minutes pour les données tardives
<code>chaos_ratio</code>	float	0.20	Fraction de trajets corrompus (utilisé en mode

Paramètre	Type	Défaut	Description
			<code>chaos</code>)
<code>seed</code>	int	42	Graine aléatoire pour la reproductibilité
<code>gold_stats_table</code>	str	auto	Table Gold de Spark Core pour calibrer les distributions

Modes de génération

normal

Distribution réaliste calquée sur le dataset NYC Taxi historique. Les zones populaires restent populaires, les montants respectent les plages observées, les clusters K-Means restent reconnaissables.

```
config = {
    "mode": "normal",
    "rate": 20,
    "interval_seconds": 30
}
```

skewed

80% des trajets proviennent d'une seule zone (LocationID 132, JFK par défaut). Force un Data Skew volontaire, utile pour tester la robustesse de vos agrégations.

```
config = {
    "mode": "skewed",
    "skew_location_id": 132, # JFK
    "skew_ratio": 0.80
}
```

chaos

Injecte `chaos_ratio` fraction de trajets corrompus selon 6 types de corruption : montants négatifs, distance nulle, distance impossible (9999), LocationID manquant, timestamps inversés (pickup après dropoff), montant aberrant (600 à 2000\$). Utile pour tester la quarantaine.

Note : les timestamps inversés ne sont pas couverts par les flags de quarantaine standards (`MONTANT_NEGATIF` , `DISTANCE_NULLE` , etc.). Les détecter via `tpep_dropoff_datetime < tpep_pickup_datetime` est une piste optionnelle pour les étudiants qui veulent couvrir tous les cas.

```
config = {  
    "mode": "chaos",  
    "chaos_ratio": 0.20  
}
```

airport_surge

Surcharge de la zone aéroport : le volume de trajets vers/depuis JFK et LGA est multiplié par 4. Simule un événement exceptionnel (vol retardé, concert). Utilisé pour le challenge de détection de drift (Séance 4).

```
config = {  
    "mode": "airport_surge",  
    "surge_multiplier": 4.0  
}
```

night

Distribution nocturne : volume global réduit, trajets plus longs, zones résider plus actives, moins de pourboires. Utile pour tester la détection de changement de pattern.

Utilisation type dans un notebook

```

import threading
import sys
sys.path.insert(0, "/Volumes/workspace/tp_spark_XYZ/scripts/")
from taxi_simulator import TaxiStreamSimulator

config = {
    "output_path": "/Volumes/workspace/tp_spark_XYZ/raw_data/stream_input/",
    "rate": 20,
    "interval_seconds": 30,
    "mode": "normal",
    "late_data_ratio": 0.10,
    "duration_batches": 20,
    "seed": 42
}

sim = TaxiStreamSimulator(**config)
thread = threading.Thread(target=sim.run, daemon=True)
thread.start()
print(f"Simulateur actif. Mode : {config['mode']}, Rate : {config['rate']}/batch")

```

Le thread `daemon=True` s'arrête automatiquement si le notebook est détaché du cluster.

Format des fichiers générés

Chaque batch génère un fichier Parquet nommé `batch_YYYYMMDD_HHMMSS.parquet` dans `output_path`.

Schéma compatible avec le dataset NYC Taxi historique :

Colonne	Type	Description
<code>VendorID</code>	integer	1 ou 2

Colonne	Type	Description
<code>tpep_pickup_datetime</code>	timestamp	Heure de prise en charge (Event Time)
<code>tpep_dropoff_datetime</code>	timestamp	Heure de déposé
<code>passenger_count</code>	double	Nombre de passagers
<code>trip_distance</code>	double	Distance en miles
<code>PULocationID</code>	integer	Zone de départ
<code>DOLocationID</code>	integer	Zone d'arrivée
<code>payment_type</code>	integer	1=carte, 2=cash
<code>fare_amount</code>	double	Tarif de base
<code>tip_amount</code>	double	Pourboire
<code>total_amount</code>	double	Total
<code>sim_batch_id</code>	integer	Numéro du batch (ajouté par le simulateur)
<code>sim_is_corrupted</code>	boolean	<code>true</code> si donnée injectée en mode chaos

La colonne `sim_is_corrupted` permet de valider votre système de quarantaine : vous pouvez vérifier a posteriori si vous avez bien détecté toutes les lignes corrompues.

Régler les temps de décalage des données tardives

Par défaut, le simulateur injecte un retard aléatoire entre `late_data_min_minutes` (1 minute) et `late_data_max_minutes` (10 minutes) sur les lignes tardives. Ces valeurs sont cohérentes avec un watermark de 5 minutes : environ la moitié des données tardives (celles entre 5 et 10 minutes) seront hors watermark et perdues.

Si vous voulez tester un comportement différent, ajustez ces deux paramètres :

```
# Toutes les données tardives dans la fenêtre du watermark (aucune perte)
config = {
    "late_data_ratio": 0.15,
    "late_data_min_minutes": 1,
```

```

    "late_data_max_minutes": 4,    # tout sous le watermark de
5 min
}

# Toutes les données tardives hors watermark (perte maximale)
config = {
    "late_data_ratio": 0.15,
    "late_data_min_minutes": 6,    # tout au-delà du watermark
de 5 min
    "late_data_max_minutes": 15,
}

# Distribution équilibrée autour du watermark (cas TP S2 par
défaut)
config = {
    "late_data_ratio": 0.15,
    "late_data_min_minutes": 1,
    "late_data_max_minutes": 10,
}

```

La règle générale : si `late_data_max_minutes <= watermark`, aucune donnée n'est perdue. Si `late_data_min_minutes > watermark`, toutes les données tardives sont perdues. Entre les deux, la fraction perdue dépend de la distribution du retard dans l'intervalle `[min, max]`.

Nettoyage entre les séances

Les fichiers Parquet s'accumulent dans `stream_input/`. Si vous ne les nettoyez pas, le prochain job Auto Loader les relira (selon le checkpoint).

```

# Supprimer les fichiers de simulation précédents
import subprocess
result = subprocess.run(
    ["rm", "-rf",
    "/Volumes/workspace/tp_spark_XYZ/raw_data/stream_inpu

```

```

t/"],
    capture_output=True, text=True
)
print("Nettoyage :", result.returncode == 0)

# Et supprimer le checkpoint correspondant si vous recommence
z à zéro
result2 = subprocess.run(
    ["rm", "-rf",
     "/Volumes/workspace/tp_spark_XYZ/checkpoints/bronze_stre
am/"],
    capture_output=True, text=True
)
print("Checkpoint effacé :", result2.returncode == 0)

```

Si vous supprimez le checkpoint sans supprimer les fichiers (ou l'inverse), votre job va tenter de relire des fichiers déjà traités ou de sauter des fichiers. Nettoyez toujours les deux ensemble.

Limites connues

Performance. Sur un compte gratuit, ne dépassez pas `rate=200` avec `interval_seconds=30`. Au-delà, le thread de génération peut lui-même devenir un goulot d'étranglement.

Reproductibilité. Avec `seed=42`, les données générées sont identiques d'une exécution à l'autre. Changez le seed si vous voulez de la variété entre les tests.

Mode chaos et quarantaine. `sim_is_corrupted` est un oracle de validation. Ne l'utilisez pas comme feature dans votre pipeline de quarantaine (vous trichez). Utilisez-le uniquement pour mesurer le recall de votre détection a posteriori.

Script complet : `taxi_simulator.py`

Copiez ce script dans votre Volume :

```
/Volumes/workspace/tp_spark_XYZ/scripts/taxi_simulator.py
```



```

"""
taxi_simulator.py
Générateur de trajets taxis pour Spark Streaming.
Déposez ce fichier dans /Volumes/workspace/tp_spark_XYZ/scripts/

Usage dans un notebook Databricks :
    import sys
    sys.path.insert(0, "/Volumes/workspace/tp_spark_XYZ/scripts/")
    from taxi_simulator import TaxiStreamSimulator
    import threading

    sim = TaxiStreamSimulator(
        output_path="/Volumes/workspace/tp_spark_XYZ/raw_data/stream_input/",
        rate=20,
        interval_seconds=30,
        mode="normal",
        duration_batches=20,
        seed=42
    )
    t = threading.Thread(target=sim.run, daemon=True)
    t.start()
"""

import os
import datetime
import threading

import pandas as pd
import numpy as np

# -----

```

```

-----
# Distributions de référence (calibrées sur NYC Taxi 2022-202
4)
# -----
-----

# Zones : (LocationID, poids de tirage, borough)
LOCATION_POOL = [
    (132, 0.08, "Queens"),      # JFK Airport
    (138, 0.06, "Queens"),      # LaGuardia Airport
    (161, 0.05, "Manhattan"),   # Midtown
    (162, 0.05, "Manhattan"),   # Midtown
    (237, 0.04, "Manhattan"),   # Upper East
    (236, 0.04, "Manhattan"),   # Upper West
    (170, 0.03, "Manhattan"),   # Murray Hill
    (48, 0.03, "Manhattan"),    # Clinton East
    (142, 0.03, "Manhattan"),   # Lincoln Square
    (79, 0.03, "Manhattan"),    # East Village
    (261, 0.03, "Manhattan"),   # World Trade Center
    (186, 0.02, "Manhattan"),   # Penn Station
    (230, 0.02, "Manhattan"),   # Times Square
    (68, 0.02, "Manhattan"),    # East Chelsea
    (100, 0.02, "Manhattan"),   # Garment District
    (90, 0.02, "Brooklyn"),     # Flatbush
    (80, 0.02, "Brooklyn"),     # East NY
    (17, 0.02, "Brooklyn"),     # Bedford
    (225, 0.02, "Queens"),      # Sunnyside
    (7, 0.02, "Queens"),        # Astoria
    (40, 0.01, "Bronx"),
    (42, 0.01, "Bronx"),
    (255, 0.01, "Staten Island"),
    (251, 0.01, "Staten Island"),
]

LOC_IDS = [x[0] for x in LOCATION_POOL]
LOC_WEIGHTS = np.array([x[1] for x in LOCATION_POOL])
LOC_WEIGHTS = LOC_WEIGHTS / LOC_WEIGHTS.sum()

```

```

LOC_BOROUGH = {x[0]: x[2] for x in LOCATION_POOL}

# Paramètres de distribution des montants (en $)
FARE_MEAN, FARE_STD = 14.5, 9.0
TIP_RATIO_MEAN = 0.18 # pourboire moyen = 18% du far
e
TIP_RATIO_STD = 0.08
DIST_MEAN, DIST_STD = 3.2, 3.5

# Paramètres de distribution temporelle
# Heure de pointe : 7-10h et 17-20h
HOUR_WEIGHTS = np.array([
    0.5, 0.3, 0.2, 0.2, 0.2, 0.4, # 0-5h
    0.8, 1.8, 2.0, 1.6, 1.2, 1.3, # 6-11h
    1.4, 1.4, 1.3, 1.4, 1.8, 2.2, # 12-17h
    2.0, 1.6, 1.3, 1.0, 0.8, 0.6, # 18-23h
])
HOUR_WEIGHTS = HOUR_WEIGHTS / HOUR_WEIGHTS.sum()

# -----
# -----
# Générateur de trajets
# -----
# -----

class TripGenerator:
    def __init__(self, rng: np.random.Generator):
        self.rng = rng

    def _pick_location(self, weights=None):
        w = weights if weights is not None else LOC_WEIGHTS
        idx = self.rng.choice(len(LOC_IDS), p=w)
        return LOC_IDS[idx]

    def _random_datetime(self, base_dt: datetime.datetime,

```

```

        hour_weights=None) -> datetime.datetime:
        """Tirage d'un datetime autour de base_dt avec distribution horaire réaliste.

        Si `hour_weights` est fourni, il remplace HOUR_WEIGHTS – utile pour
        contraindre l'heure (mode night, etc.).
        """
        w = hour_weights if hour_weights is not None else HOUR_WEIGHTS
        hour = int(self.rng.choice(len(w), p=w))
        minute = int(self.rng.integers(0, 60))
        second = int(self.rng.integers(0, 60))
        return base_dt.replace(hour=hour, minute=minute, second=second, microsecond=0)

    def generate(self, n: int, base_dt: datetime.datetime,
                 pu_weights=None, do_weights=None,
                 hour_weights=None) -> list[dict]:
        rows = []
        for _ in range(n):
            pu_loc = self._pick_location(pu_weights)
            do_loc = self._pick_location(do_weights)
            pickup = self._random_datetime(base_dt, hour_weights=hour_weights)
            dist = max(0.1, float(self.rng.normal(DIST_MEAN, DIST_STD)))
            duration_min = max(1, dist * 4 + float(self.rng.normal(0, 3)))
            dropoff = pickup + datetime.timedelta(minutes=duration_min)
            fare = max(2.5, float(self.rng.normal(FARE_MEAN, FARE_STD)))
            tip_r = max(0.0, float(self.rng.normal(TIP_RATIO_MEAN, TIP_RATIO_STD)))

```

```

        tip      = round(fare * tip_r, 2)
        total    = round(fare + tip + 0.5 + 0.3, 2) # sur
charges fixes
        rows.append({
            "VendorID":                int(self.rng.choice
([1, 2])),
            "tpep_pickup_datetime":    pickup,
            "tpep_dropoff_datetime":   dropoff,
            "passenger_count":          float(self.rng.choic
e([1, 1, 1, 2, 2, 3, 4], p=[0.5, 0.15, 0.15, 0.1, 0.05, 0.03,
0.02])),
            "trip_distance":            round(dist, 2),
            "PULocationID":             pu_loc,
            "DOLocationID":             do_loc,
            "payment_type":              int(self.rng.choice
([1, 2], p=[0.7, 0.3])),
            "fare_amount":               round(fare, 2),
            "tip_amount":                tip,
            "total_amount":              total,
            "sim_is_corrupted":          False,
        })
    return rows

```

```

# -----
# Injecteur de chaos
# -----

```

```

class ChaosInjector:
    def __init__(self, rng: np.random.Generator, chaos_ratio:
float = 0.20):
        self.rng = rng
        self.chaos_ratio = chaos_ratio
        self._corruptions = [

```

```

        self._negative_amount,
        self._zero_distance,
        self._impossible_distance,
        self._null_location,
        self._inverted_timestamps,
        self._huge_amount,
    ]

    def inject(self, rows: list[dict]) -> list[dict]:
        n_corrupt = max(1, int(len(rows) * self.chaos_ratio))
        indices = self.rng.choice(len(rows), size=n_corrupt,
replace=False)
        for idx in indices:
            corruption = self._corruptions[int(self.rng.integers(len(self._corruptions)))]
            rows[idx] = corruption(rows[idx])
            rows[idx]["sim_is_corrupted"] = True
        return rows

    def _negative_amount(self, row):
        row["total_amount"] = -abs(row["total_amount"])
        row["fare_amount"] = -abs(row["fare_amount"])
        return row

    def _zero_distance(self, row):
        row["trip_distance"] = 0.0
        return row

    def _impossible_distance(self, row):
        row["trip_distance"] = 9999.0
        return row

    def _null_location(self, row):
        row["PULocationID"] = None
        return row

```

```

def _inverted_timestamps(self, row):
    row["tpep_pickup_datetime"], row["tpep_dropoff_datetime"] = (
        row["tpep_dropoff_datetime"],
        row["tpep_pickup_datetime"],
    )
    return row

def _huge_amount(self, row):
    row["total_amount"] = float(self.rng.integers(600, 2000))
    row["fare_amount"] = row["total_amount"] * 0.85
    return row

# -----
# Simulateur principal
# -----

class TaxiStreamSimulator:
    """
    Génère des fichiers Parquet imitant des trajets taxis en
    temps réel.

    Paramètres
    -----
    output_path : str
        Chemin Unity Catalog où écrire les fichiers.
        Ex : /Volumes/workspace/tp_spark_XYZ/raw_data/stream_
input/
    rate : int
        Nombre de trajets par batch (défaut : 20).
    interval_seconds : int
        Secondes entre deux batches (défaut : 30).

```

```

mode : str
    normal | skewed | chaos | airport_surge | night | drift
duration_batches : int
    Nombre de batches avant arrêt automatique (défaut : 20).
late_data_ratio : float
    Fraction de trajets avec Event Time en retard (0.0 à 0.5).
late_data_min_minutes : int
    Retard minimum en minutes (défaut : 1).
late_data_max_minutes : int
    Retard maximum en minutes (défaut : 10).
chaos_ratio : float
    Fraction de trajets corrompus en mode chaos (défaut : 0.20).
skew_location_id : int
    LocationID sur-représenté en mode skewed (défaut : 132 = JFK).
skew_ratio : float
    Proportion du trafic sur la zone skewée (défaut : 0.80).
surge_multiplier : float
    Multiplicateur de volume aéroport en mode airport_surge (défaut : 4.0).
seed : int
    Graine aléatoire pour la reproductibilité (défaut : 42).
verbose : bool
    Afficher les logs de chaque batch (défaut : True).
"""

def __init__(
    self,
    output_path: str,
    rate: int = 20,

```



```

interval_seconds: int = 30,
mode: str = "normal",
duration_batches: int = 20,
late_data_ratio: float = 0.0,
late_data_min_minutes: int = 1,
late_data_max_minutes: int = 10,
chaos_ratio: float = 0.20,
skew_location_id: int = 132,
skew_ratio: float = 0.80,
surge_multiplier: float = 4.0,
seed: int = 42,
verbose: bool = True,
):
    self.output_path = output_path
    self.rate = rate
    self.interval_seconds = interval_seconds
    self.mode = mode
    self.duration_batches = duration_batches
    self.late_data_ratio = late_data_ratio
    self.late_data_min_minutes = late_data_min_minutes
    self.late_data_max_minutes = late_data_max_minutes
    self.chaos_ratio = chaos_ratio
    self.skew_location_id = skew_location_id
    self.skew_ratio = skew_ratio
    self.surge_multiplier = surge_multiplier
    self.seed = seed
    self.verbose = verbose

    self.rng = np.random.default_rng(seed)
    self.generator = TripGenerator(self.rng)
    self.chaos = ChaosInjector(self.rng, chaos_ratio)
    self._batch_id = 0
    self._stop_event = threading.Event()

    os.makedirs(output_path, exist_ok=True)

```

```

# -----
# Modes de génération
# -----

def _generate_normal(self, base_dt):
    return self.generator.generate(self.rate, base_dt)

def _generate_skewed(self, base_dt):
    n_skew = int(self.rate * self.skew_ratio)
    n_normal = self.rate - n_skew
    # Poids concentrés sur la zone cible
    skew_w = np.zeros(len(LOC_IDS))
    if self.skew_location_id in LOC_IDS:
        skew_w[LOC_IDS.index(self.skew_location_id)] = 1.
    else:
        skew_w[0] = 1.0
    rows = self.generator.generate(n_skew, base_dt, pu
_weights=skew_w)
    rows += self.generator.generate(n_normal, base_dt)
    return rows

def _generate_chaos(self, base_dt):
    rows = self.generator.generate(self.rate, base_dt)
    return self.chaos.inject(rows)

def _generate_airport_surge(self, base_dt):
    # JFK (132) et LGA (138) surgent
    surge_ids = [132, 138]
    surge_w = LOC_WEIGHTS.copy()
    for sid in surge_ids:
        if sid in LOC_IDS:
            surge_w[LOC_IDS.index(sid)] *= self.surge_mul
    multiplier

```

```

        surge_w = surge_w / surge_w.sum()
        return self.generator.generate(self.rate, base_dt, pu
_weights=surge_w)

    def _generate_drift(self, base_dt):
        # Transition linéaire normal -> airport_surge sur dur
        ation_batches.
        progress = self._batch_id / max(1, self.duration_batc
        hes - 1)
        mult = 1.0 + (self.surge_multiplier - 1.0) * progress
        w = LOC_WEIGHTS.copy()
        for sid in (132, 138):
            if sid in LOC_IDS:
                w[LOC_IDS.index(sid)] *= mult
        w = w / w.sum()
        return self.generator.generate(self.rate, base_dt, pu
_weights=w)

    def _generate_night(self, base_dt):
        # Heure forcée entre 22h et 4h, trajets plus longs
        night_w = np.zeros(24)
        for h in (22, 23, 0, 1, 2, 3):
            night_w[h] = 1.0
        night_w = night_w / night_w.sum()
        rows = self.generator.generate(
            max(1, self.rate // 2), base_dt, hour_weights=nig
            ht_w
        )
        # Allonger les trajets
        for r in rows:
            r["trip_distance"] = round(r["trip_distance"] *
            1.6, 2)
            r["total_amount"] = round(r["total_amount"] *
            1.4, 2)
        return rows

```

```

# -----
-----

# Injection de retard (late data)
# -----
-----

def _apply_late_data(self, rows, base_dt):
    if self.late_data_ratio <= 0:
        return rows
    n_late = max(1, int(len(rows) * self.late_data_ratio))
    indices = self.rng.choice(len(rows), size=n_late, replace=False)
    lo = max(1, self.late_data_min_minutes)
    hi = max(lo, self.late_data_max_minutes)
    for idx in indices:
        delay = int(self.rng.integers(lo, hi + 1))
        for col in ("tpep_pickup_datetime", "tpep_dropoff_datetime"):
            rows[idx][col] = rows[idx][col] - datetime.timedelta(minutes=delay)
    return rows

# -----
-----

# Écriture Parquet
# -----
-----

def _write_batch(self, rows: list[dict]):
    ts = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")

    filename = f"batch_{ts}_{self._batch_id:04d}.parquet"
    filepath = os.path.join(self.output_path, filename)

    df = pd.DataFrame(rows)

```

```

        df["sim_batch_id"] = self._batch_id

        # Cast explicite des types pour compatibilité Spark
        df["VendorID"] = df["VendorID"].astype("int32")
        df["PULocationID"] = pd.array(df["PULocationID"], dtype="Int32") # nullable int
        df["DOLocationID"] = df["DOLocationID"].astype("int32")
        df["payment_type"] = df["payment_type"].astype("int32")
        df["passenger_count"] = df["passenger_count"].astype("float64")
        df["trip_distance"] = df["trip_distance"].astype("float64")
        df["fare_amount"] = df["fare_amount"].astype("float64")
        df["tip_amount"] = df["tip_amount"].astype("float64")
        df["total_amount"] = df["total_amount"].astype("float64")
        df["sim_batch_id"] = df["sim_batch_id"].astype("int32")
        df["sim_is_corrupted"] = df["sim_is_corrupted"].astype("bool")

        df.to_parquet(filepath, index=False, engine="pyarrow")

        return filepath, len(df)

    # -----
    # Boucle principale
    # -----

def stop(self):

```

```

        """Demande l'arrêt propre du simulateur (effectif après le batch courant)."""
        self._stop_event.set()

    def run(self):
        """
        Lance la simulation. À appeler dans un thread daemon.
        Arrêt automatique après duration_batches batches, ou
        via stop().
        """
        self._stop_event.clear()
        generators = {
            "normal": self._generate_normal,
            "skewed": self._generate_skewed,
            "chaos": self._generate_chaos,
            "airport_surge": self._generate_airport_surge,
            "night": self._generate_night,
            "drift": self._generate_drift,
        }

        gen_fn = generators.get(self.mode)
        if gen_fn is None:
            raise ValueError(
                f"Mode inconnu : '{self.mode}'. "
                f"Valeurs acceptées : {list(generators.keys())}"
            )

        if self.verbose:
            print(
                f"[Simulateur] Démarrage – mode={self.mode}, "
                f"rate={self.rate}, interval={self.interval_s} "
                f"econds}s, "
                f"batches={self.duration_batches}"
            )

```

```

for batch_num in range(self.duration_batches):
    if self._stop_event.is_set():
        break
    self._batch_id = batch_num
    base_dt = datetime.datetime.now()

    rows = gen_fn(base_dt)
    rows = self._apply_late_data(rows, base_dt)

    filepath, n = self._write_batch(rows)

    if self.verbose:
        print(
            f"[Simulateur] Batch {batch_num + 1}/{self.duration_batches} "
            f"- {n} trajets - {os.path.basename(filepath)}"
        )

    if batch_num < self.duration_batches - 1:
        # wait() au lieu de sleep() pour répondre au
        stop() pendant l'attente
        if self._stop_event.wait(self.interval_seconds):
            break

    if self.verbose:
        print("[Simulateur] Simulation terminée.")

```